OOP techniques

Jarmo Hurri

Overview

- different constructs for object-oriented programming
- encapsulation
  - packaging data and functionality
- inheritance
- polymorphism
- aggregation
- association

Scanned with CamScanner

Encapsulation

- meaning (Oxford dictionary)
  enclosing something in or as if in a capsule
- in OOP, a class
  - encapsulates data and (related) methods
  - possibly restricting direct access to some components
- some advantages
  - denies unwanted access from clients
  - hides irrelevant (complex) details
  - simplifies maintenance (if only internals change)
  - improves *reusability* (use of code elsewhere) by making clear what services the class provides

Access control

- not all attributes need to be accessible to code that uses a class (client)
- *information hiding*: only provide access to relevant information
- *access control*: restrict access to attribute

access	UML	class itself	child class	other
public	+	✓	✓	✓
protected	#	✓	✓	
private	-	✓		

- Python approach: no strict enforcement
  - private: mechanism which can be circumvented
  - protected: guideline

Scanned with CamScanner

Attribute access control in Python

- public: default type
- private
  - mechanism
  - identifier begins with double underscore (e.g. `__priv_attr`) but does not end with double underscore
  - identifier visible outside class *mangled* by Python (e.g. `_ClassName__priv_attr`)
  - attribute can not be accessed accidentally
  - but mechanism can be circumvented
  - access can be provided by defining
    - *accessor*: method returning value
    - *mutator*: method setting value
- protected
  - guideline (no mechanism)
  - name begins with single underscore (e.g. `_prot_attr`)

Example 1 ↪

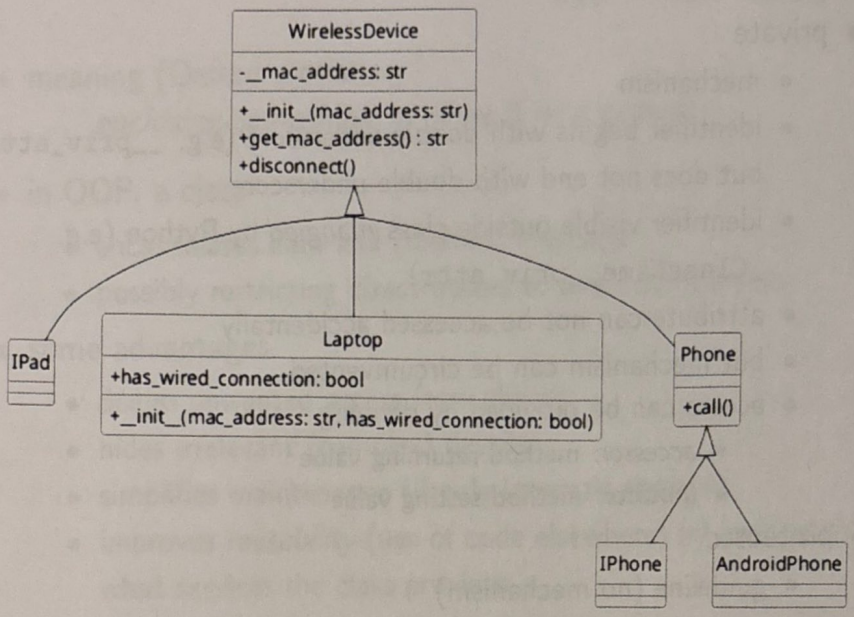
```
class Cls:
    def __init__(self):
        self.__foo = 'foo value'
        self._bar = 'bar value'
    def get_foo(self): return self.__foo # accessor
    def set_foo(self, foo): self.__foo = foo # mutator

x = Cls()
try:
    print(x.__foo) # this will not work
except Exception as e:
    print('exception:', e)

# but these will
print(x.get_foo()) # using accessor
x.set_foo('new foo value') # using mutator
print(x._Cls__foo) # circumventing mangling
print(x._bar) # direct access to protected
```


Scanned with CamScanner

Example 2: inheritance



Inheritance

- "is a"-relationship
  - an iPhone is a phone is a wireless device
- definition

  Parent object holds data and actions for related child objects.
- static: does not change over program lifetime
- *child* class (or *subclass*, or *derived class*) inherits data and methods of *parent* class (or *superclass*, or *base class*)
- related terminology: abstraction / generalization (upwards) and concretization / specialization (downwards)

Scanned with CamScanner

Inheritance in Python

- list parent class(es) after name of class in parentheses
- if parent needs initialization, its initializer is available as `super().__init__()`.
- for example ↪

```
class WirelessDevice:
    def __init__(self, mac_address):
        self.__mac_address = mac_address

    def get_mac_address(self): return self.__mac_address

class Laptop(WirelessDevice):
    def __init__(self,
                 mac_address,
                 has_wired_connection):
        # call superclass constructor
        super().__init__(mac_address)
        self.has_wired_connection = has_wired_connection;
```


Polymorphism

- both child and parent can have string representations via method `__str__()`
- child can specify its own representation: it can *override* the method of parent
- this is one case of *polymorphism*
  methods can have same name but different bodies and parameter lists
- program knows which one is executed by taking a look at class of reference *overriding* (subtype polymorphism) parameters *overloading* (ad hoc polymorphism, not in Python)
- if child needs corresponding method of parent, child can use it via `super()`

Scanned with CamScanner

Example 3 ↪

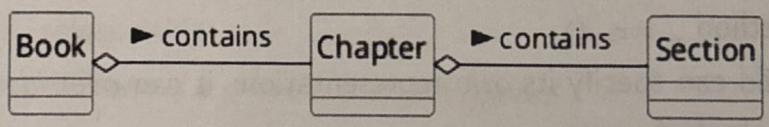
```
class Parent:
    def __str__(self):
        return 'parent'

class Child(Parent):
    def __str__(self):
        return super().__str__() + ' & child'

p = Parent() # superclass object
c = Child() # subclass object
print(p) # call __str__()
print(c)
```

```
parent
parent & child
```

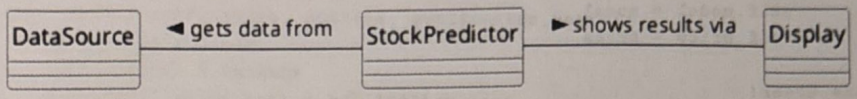

Aggregation



- "has a"-relationship
- dynamic: objects can change during program lifetime
- does not imply absolute ownership
  - same chapter might be in another book
- can be implemented with
  - reference variable if has one one or zero objects
  - list in case of multiple objects

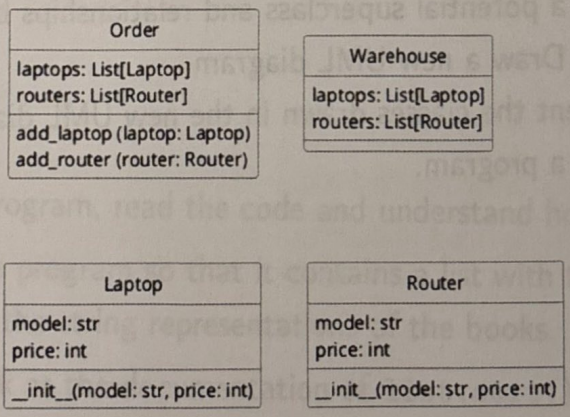
Scanned with CamScanner

Association



- very general "uses a"-relationship
- dynamic: objects can change during program lifetime
- implementation with reference variable or list

Example 4



- computer store
  - sells laptops and routers
  - processes orders for these
  - has a warehouse (stock)

Scanned with CamScanner

Example continues ↪

```
class Laptop:
    def __init__(self, model, price):
        self.model = model
        self.price = price

class Router:
    def __init__(self, model, price):
        self.model = model
        self.price = price

class Order:
    def __init__(self):
        self.laptops = []
        self.routers = []

    def add_laptop(self, laptop): self.laptops.append(laptop)

    def add_router(self, router): self.routers.append(router)

class Warehouse:
    def __init__(self, laptops, routers):
        self.laptops = laptops
        self.routers = routers

warehouse = Warehouse([Laptop('mac', 1999)], [Router('netgate', 600)])
```


Example continues

1. Identify a potential superclass and relationships between classes. Draw a new UML diagram.
2. Implement the classes drawn in the new UML diagram and use them in a program.

Scanned with CamScanner

Homework 1

Consider the following program. ↪

```
class Book:
    def __init__(self, title, authors, publication_year):
        self.title = title
        self.authors = authors
        self.publication_year = publication_year

    def __str__(self):
        auth = ' & '.join(self.authors)
        return f'{auth}: {self.title} ({self.publication_year})'

    def num_authors(self):
        return len(self.authors)

clrs = Book('Introduction to Algorithms (3rd ed.)',
            ['Cormen', 'Leiserson', 'Rivest', 'Stein'],
            2009)
print(clrs)
```

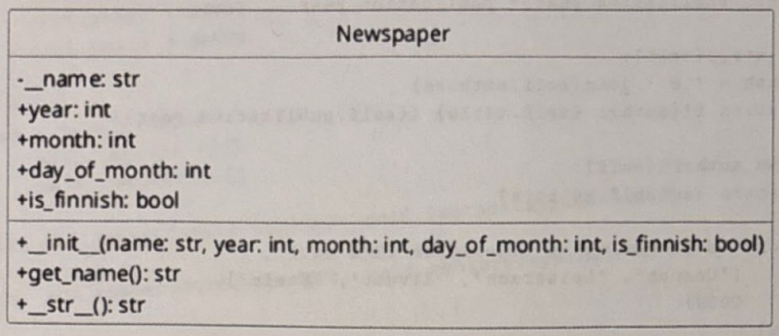

Homework continues

1. Run the program, read the code and understand how it works.
2. Modify the program so that it contains a list with three books and prints the string representations of the books.
3. Take a look at the documentation of `list.sort()`. Use the optional parameters `key` and `reverse` to sort your list of books in descending order according to number of authors.

Scanned with CamScanner

Homework 2

Implement in Python the class shown in the UML diagram, and use it in a program. You can freely choose the string representation of objects of this class.



Homework 3

Draw an UML diagram and write and use the corresponding Python class for class `Course`, which has a

- name
- topic
- name of teacher
- number of participants.

Scanned with CamScanner

Homework 4

Write a Python class `Point` for a point in the xy-plane which, in addition to initialization and string representation, supports the following operations:

- determining whether the point is on the x-axis
- determining whether the point has the same coordinates as another point [note: Python calls dunder method `__eq__(self,value)` automatically when operator == is used]
- finding the distance from another point
- determining whether the point is in the unit circle (including its boundary)
- finding the midpoint of the line formed by the point and another point.

Use your class in a program.

Homework 5

Construct an UML diagram with class relationships for appropriately named classes for concepts

- xy-plane
- triangle
- line segment.

Introduce one common superclass for some of these classes.

Scanned with CamScanner

Homework 6

In a company, each project has a unique integer identifier and consists of one or more tasks. Each task has a name and is completed by one or more worker. Each worker has a name, and can be either an employee or a hired contractor.

1. Draw an UML diagram for these classes.
2. Write a Python implementation of the diagram. Use the classes in a program to create and print a project with two tasks and two workers.

© Jarmo Hurri

Scanned with CamScanner